FIELD NOTES ON AI ASSISTED SOFTWARE DEVELOPMENT

Beyond the Prompt: Engineering Your Codex Workflow

Lessons learned / Live workflow demos / What comes next

What changed in my workflow

More capable agents

Less procedural handholding

Clear requirements still matter

Instructions and reusable knowledge

AGENTS.md

Project essentials, boundaries, key commands, and pointers.

Documentation

Detailed context where the relevant task needs it.

Skills

Specialized procedures worth repeating.

Test-first feedback

Define the expected behavior

See the test fail for the right reason

Implement, then verify and refactor

A failing test makes the gap visible before the fix.

Projects, threads, and tasks

Project

The working context and shared files.

Thread

A focused conversation for a piece of work.

Task

The outcome to deliver.

Hooks: automatic checks

A lifecycle event starts a check

The check returns evidence

A failure gives Codex a reason to continue

Rules: command boundaries

Allow

Commands that run under the configured policy.

Ask

Commands that need approval, such as git push.

Forbid

Commands blocked by policy, such as git reset.

Skills: a reusable workflow

A clear trigger

A repeatable procedure

References and tools the workflow needs

Sub-agents: delegated work

Planner

Sets acceptance criteria and bounded tasks.

Implementers

Work in parallel in explicitly separate worktrees.

Reviewer

Integrates, fixes issues, and verifies the result.

Plan / Failing tests / Implementation / Review

Intent and focused context

State the outcome and boundaries

Resolve unknowns before implementation

Carry decisions into project files

Verification and autonomy

Run relevant checks during development

Run broader checks at integration points

Match model and reasoning to task complexity

Product-minded engineering

Understand the problem

Make informed trade-offs

Own the outcome

Better agents expand what we can build. Our judgment shapes what is worth building.